WEEK 1 TEAM PROJECT – TEAM SENTINEL

01 Executive Summary

We built a small, isolated IT environment in Virtualbox made up of a windows 10 endpoint, an ubuntu server 22.04.5 and a Kali linux tester, all on one private NAT Network (10.0.2.0/24). we recorded each machine's identity and network settings, took a baseline of the windows endpoint, carried out basic administration on the ubuntu server, and captured network traffic with Wireshark.

We then ran a small, controlled test from Kali against our machines only, a port scan and a series of failed and successful SSH logins against the ubuntu server. The activities were recorded in the ubuntu authentication log and it was traced to Kali's address, and the windows firewall hid the endpoint's services from scan. The lesson is that ordinary logs and packet capture are enough to see, attribute and reconstruct this kind of activity, provided the clocks and baseline are understood.

02 Environment Overview and network diagram

For this project we built a small IT environment inside VirtualBox on a Windows PC. It has three Virtual machines (VMs): a Windows 10 machine acting as the office endpoint, an ubuntu Server acting as the server, and a Kali Linux machine acting as the security tester. All three machines sit on one private virtual network, so they can communicate with one another but nothing in the lab is exposed to anyone else.

Role	Machine	Operating system	Purpose
Endpoint	Windows-Endpoint	Windows 10	Acts as an employee workstation that we baseline and monitor
Server	Ubuntu-server	Ubuntu server 22.04.5 LTS	Acts as a company server that we administer and collect logs from
Tester	Kali Linux	Kali Linux 2026.2(pre-built Virtualbox image)	Used later for controlled security activity against our own lab

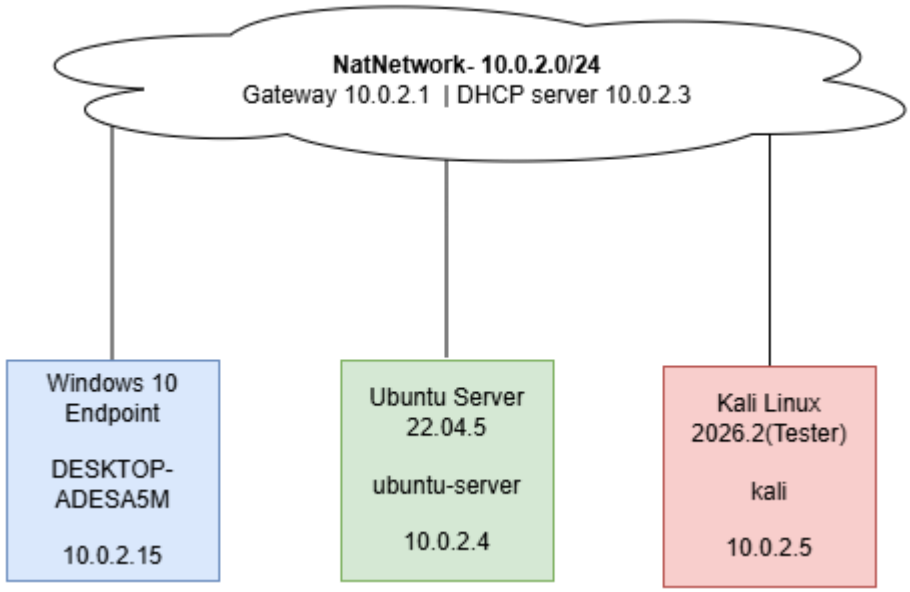
How each machine was set up

Windows 10 endpoint: we set up a windows virtual machine using virtualbox on a windows 10 host computer. We attached the already downloaded windows 10 ISO and ran the standard set up.

Ubuntu Server: we installed ubuntu server 22.04.5 from the official live-server ISO. During installation we used the guided storage option with the entire virtual disk. We created an administrator account and enabled OpenSSH server so the machine can be reached remotely later.

Kali Linux: We downloaded the pre-built Kali virtualbox image, which arrived as a compressed archive. We extracted it with WINRAR which produced a folder containing the machine definition file and the virtual disk image. We then attempted to open the file in virtualbox to add it. On the first try it was opened twice, so Kali showed up as two entries. We powered both off and removed the duplicate using “Remove only” , so that the original disk files were kept.

Network diagram



Connectivity checks

To prove the machines can communicate with each other, we started a ping from the ubuntu server to the windows-endpoint (10.0.2.15). Windows block the incoming ping (ICMP echo) through its firewall by default. We fixed this by adding an inbound rule from the administrator command prompt after which the ubuntu ping started returning replies (see Fig 1 & 2). We also pinged the kali Linux from the ubuntu server and also got replies. We did this for all three machines.

Summary of how we verified communication

Check	From	To	Address used	How I verified it	Result
Ping	ubuntu	windows	10.0.2.15	ping from ubuntu after allowing ICMP in the windows firewall	About 90 replies
Ping	Kali	ubuntu	10.0.2.4	Wireshark on kali with filter ICMP	Every request answered
Ping	Kali	Windows	10.0.2.15	Wireshark on Kali with filter ICMP	Every request answered
IP configuration	Each VM	Each VM	Own address	ip a (ubuntu), ipconfig/all (Windows), Wireshark (Kali)	Windows, ubuntu and Kali confirmed
Service connection	Kali	ubuntu	10.0.2.4, port 22 (SSH)	sshd entries for 10.0.2.5 in /var/log/auth.log	Connections and logins recorded

Fig 1: Inbound rule from administrator command to allow incoming ping

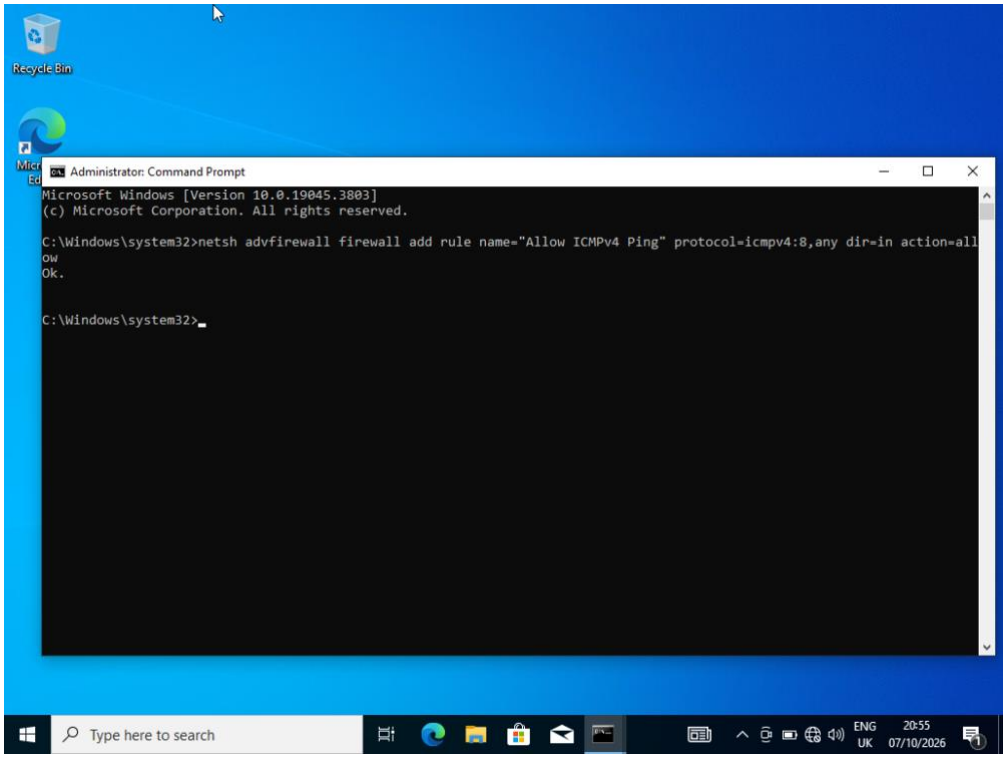


Fig 2: Successful ping to windows

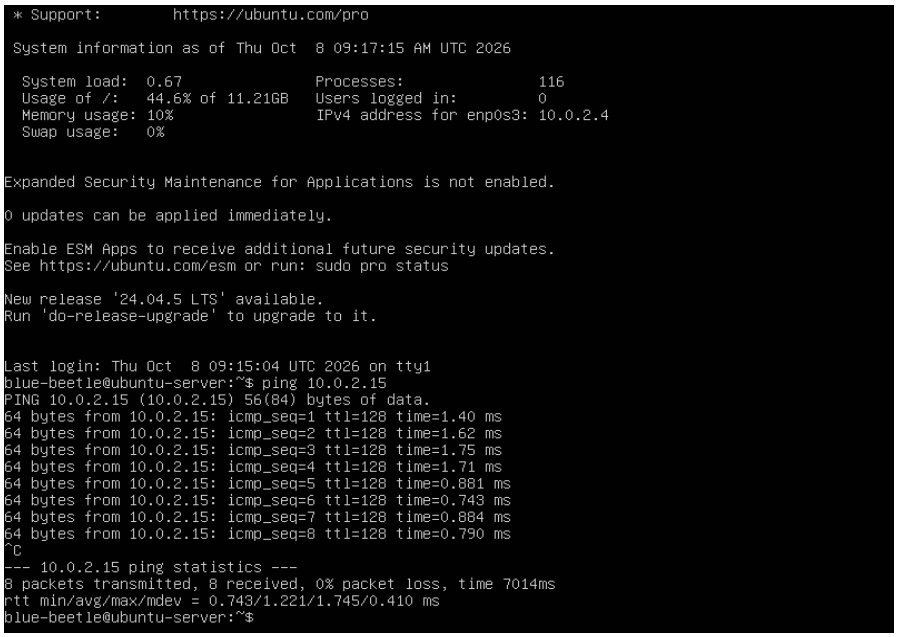
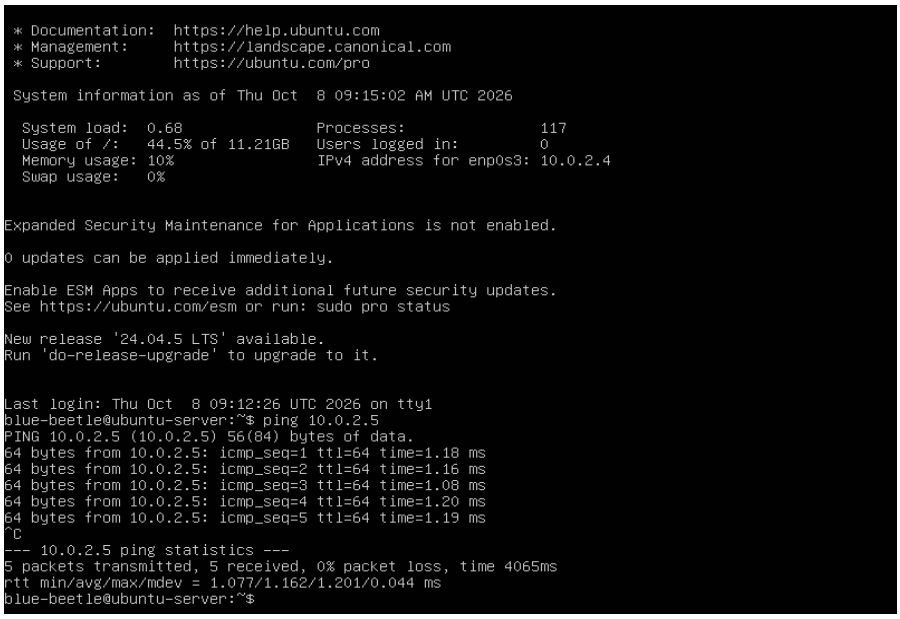


Fig 3: Successful ping to Kali Linux



Problems we ran into and how we solved them

Problems	Cause	Fix
Kali appeared twice in Virtualbox	The machine file was opened twice	Powered both off and removed the duplicate
Ubuntu and windows both showed 10.0.2.15	Ubuntu was still on plain NAT	Moved ubuntu to the NATNetwork; it received 10.0.2.4

Ping to windows and got no replies	Windows firewall blocks inbound ICMP by default	Added an inbound ICMPv4 allow rule with netsh
Ping showed “destination host unreachable” at first	Target machine was powered off	Powered on all machines and confirmed they shared the NAT Network

03.Asset and Network Inventory

Item	Windows endpoint	Ubuntu server	Kali tester
Operating system	Windows 10	Ubuntu 22.04.5 LTS	Kali Linux 2026.2
User account	Glow Ree	blue-beetle	kali
Network adapter	Intel PRO/1000 MT Desktop	Enp0s3	Intel PRO/1000 MT Desktop(82540EM)
MAC address	08:00:27:4D:39:F4	08:00:27:b6:33:b4	08:00:27:5A:87:BC
IPv4 address	10.0.2.15	10.0.2.4	10.0.2.5
Subnet mask	255.255.255.0	255.255.255.0	255.255.255.0
Hostname	DESKTOP-ADESA5M	ubuntu-server	Kali
Default gateway	10.0.2.1	10.0.2.1	10.0.2.1
DHCP server	10.0.2.3	10.0.2.3	10.0.2.3
Role	Endpoint	Server	Tester
DNS	Fec0:0:0:ffff:1, ::2, ::3		

04. Windows baseline

The windows endpoint is DESKTOP-ADESA5M, running windows 10 and we were logged in as Glow Ree. Its IP configuration is in section 3.

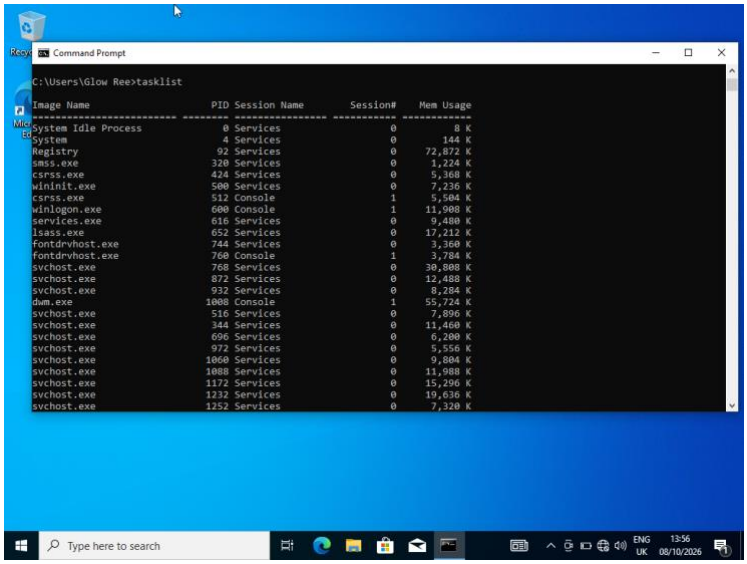
Processes I examined

We chose five processes that should be present on any windows 10 machine and wrote down what each does.

Process	What it does	Why it matters	Expected?	What would make it suspicious
lsass.exe	Checks passwords at login and builds the security token for each session	It holds credentials so attackers target it	Yes	A second copy, a wrong location or other programs trying to read its memory

Svchost.exe	A host program that runs many windows background services	Large part of the system depends on it	Yes	A copy outside C:\Windows\System32, a misspelt name, or one not started by the service manager
Winlogon.exe	Runs the sign-in, sign-out and lock screen	It controls how users enter the system	Yes	A copy in the wrong folder or extra copies
fontdrvhost.exe	Loads fonts and processes in a separate, protected process instead of inside the core of windows	It keeps a badly made font from harming the system, so it is a security safeguard	Yes	A copy outside C:\Windows\System32, or a copy that talks to the internet
csrss.exe	supports the windows environment	The system depends on it, so windows stop if it is killed	Yes	A second copy from the wrong folder, or a lookalike name such as csrs.exe

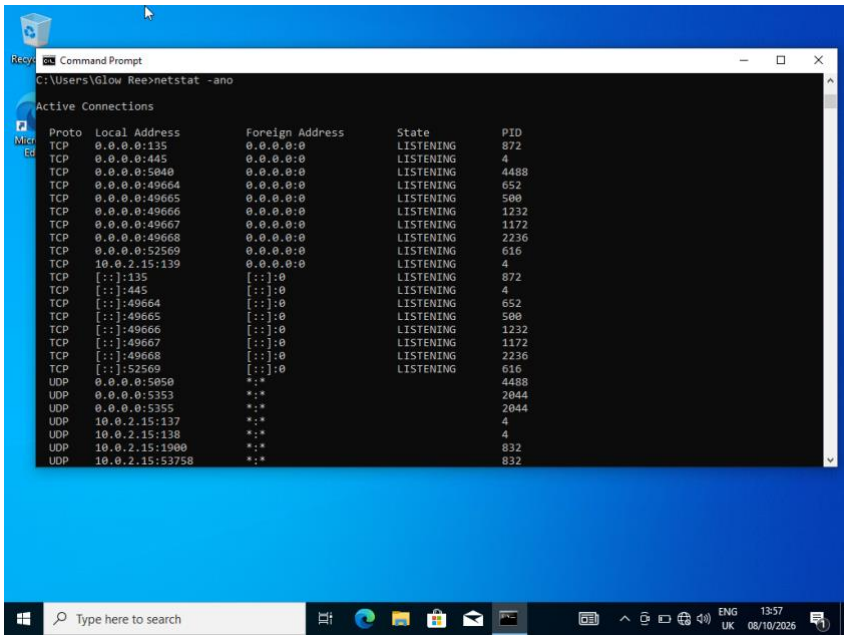
Fig 4: the list of running processes



Services, listening ports and active connections

This is the record of what the endpoint offers and what its connected to before any test. we used net start for running services and netstat -ano for listening ports and connections.

Fig 5: listening ports and connections



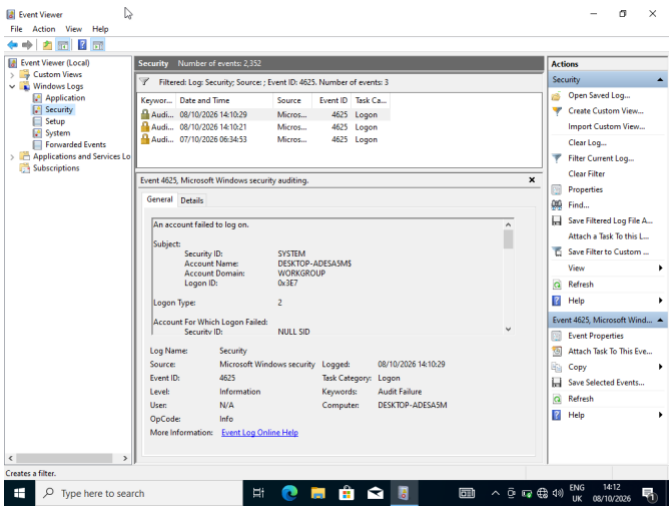
Firewall status

The windows firewall was on. We added one deliberate exception, an inbound rule allowing ICMPv4 echo(ping), so the machines could test connectivity (section 2.4). Later, an Nmap scan from Kali found no usable open ports on the machine.

Authentication and system activity

We opened event viewer on the endpoint to reach the security log, where event ID 4624 records successful logon and 4625 a failed one.

FIG 6: windows event viewer and the failed login event



05 Linux Administration

We created two users, analyst1 and analyst2, and one group, soc-team. Accounts are created with a password and a home folder, and a group lets us grant access to team instead of to individuals.

Command	What it does
sudo adduser analyst1 and sudo adduser analyst2	Creates the two users with home folders and passwords
sudo groupadd soc-team	Creates the group
id analyst1 and id analyst2	Shows each user's ID and groups

Adding and removing a group member

Command	What it does
sudo usermod -aG soc-team analyst1 and sudousermod -aG soc-team analyst2	Adds both users to soc-team
getent group soc-team	Shows the members of the group
sudo gpasswd -d analyst2 soc-team	Removes analyst2 from the group
Getent group soc-team	Shows the membership again: only analyst1 remains

Permissions test

To show how permissions work we made a shared folder owned by analyst1 and the group soc-team, with mode 770. That gives the owner and the group full access and everyone else none.

Command	What it does
sudo mkdir /srv/labshare	Creates the test folder
sudo chown analyst1 :soc-team /srv/labshare	Changes the owner to analyst1 and the group to soc-team
sudo chmod 770 /srv/labshare then ls -ld /srv/labshare	Sets the mode
sudo -u analyst1 touch /srv/labshare/a.txt	The owner creates a file
sudo -u analyst2 ls /srv/labshare	A user outside the group is refused (permission denied)

Normal and failed logins

We logged in normally at the console as blue-beetle and over SSH from Kali, and we made a few deliberate failed SSH logins from Kali against our own server. The log evidence for this is in section 06 and 08

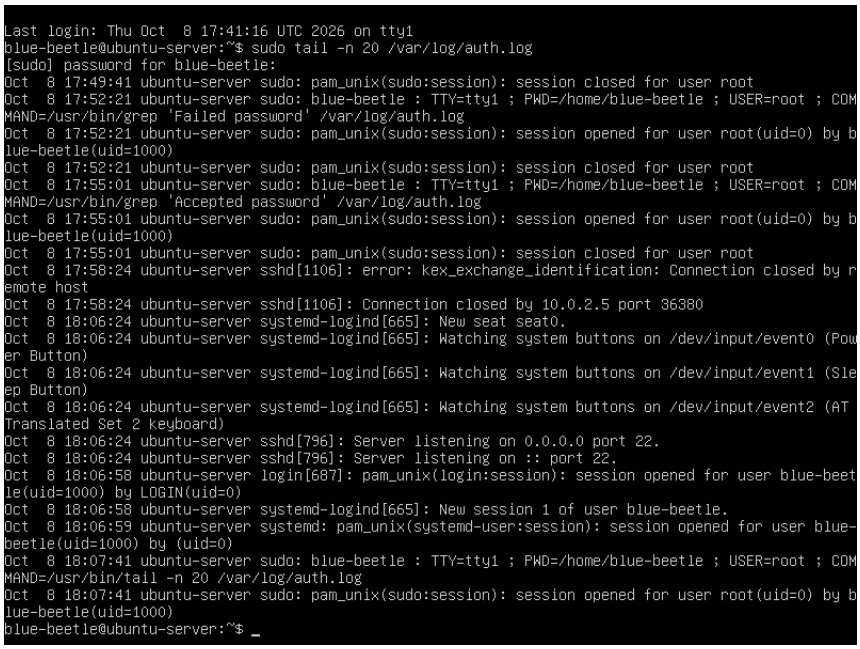
Admin action

We used sudo for privileged commands throughout. Each use was recorded with the user, terminal, working directory and exact commands.

06 Evidence and Logs

The ubuntu authentication log (/var/log/auth.log) records sudo use, logins, SSH connections and service start-up.

Fig 6: last 20 lines of /var/log/auth.log on ubuntu-sever

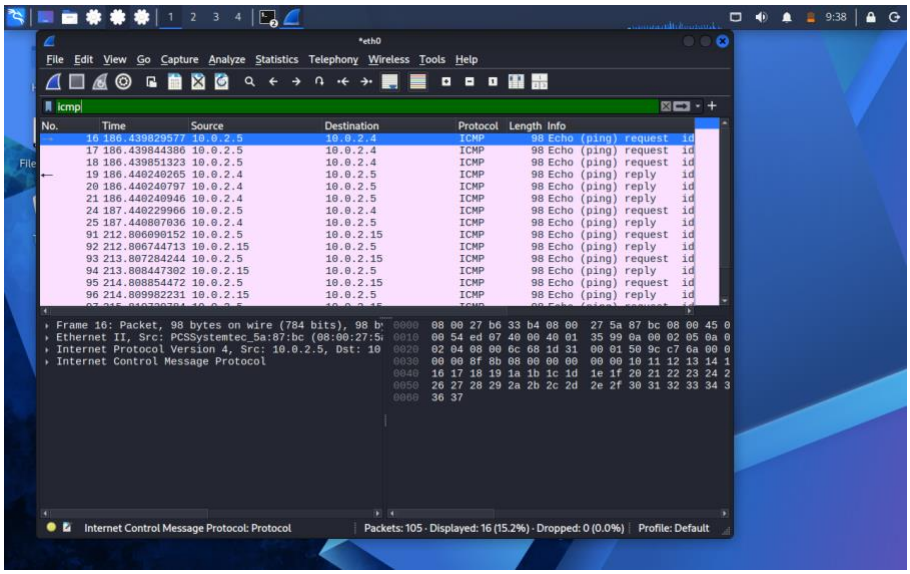


Activity	Evidence found	Where	What it told us
User creation	useradd[1021]: new user: name=analyst1, UID=1001, GID=1001, home=/home/analyst1 at Oct 8 14:15:55 UTC	/var/log/auth.log	blue-beetle created the account analyst1 from a terminal
Normal login	Session opened for user blue-beetle at 18:06:58 UTC	/var/log/auth.log	A console login by blue-beetle, and the exact time
Failed login	Failed password for blue-beetle from 10.0.2.5	/var/log/auth.log	Repeated wrong passwords from Kali's address
Permission change	chmod and chown are not written to auth.log	Command output; the sudo line for the command	The log shows who used sudo
Admin action	Sudo entries at 17:52:21, 17:55:01 and 18:07:41 UTC, with full command shown	/var/log/auth.log	Which user ran which privileged command, from which terminal, and when
Windows logon	Event IDs 4624 and 4625	Event viewer	Successful and failed logons on the endpoint, in local time

07 Wireshark Analysis

We captured traffic on Kali's eth0 interface using Wireshark, run with sudo. At first we got an idle baseline with only one capture but after we pinged the other machines the captures grew to 103 packets. It contained ICMP echo requests and replies between Kali (10.0.2.5) and Windows (10.0.2.15), ARP packets and background DHCPv6 and ICMPv6.

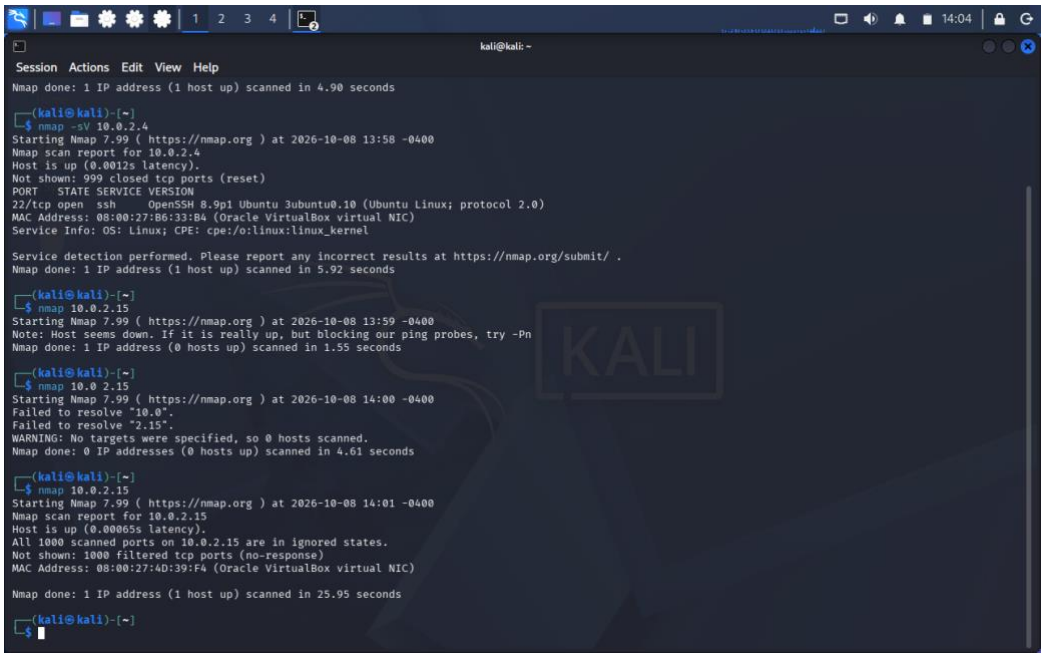
Fig 7: Wireshark with the display filter icmp



08 Controlled Security Activity

Activity	From	Against	When	why	What we observed	Evidence
Port scan of windows	Kali (10.0.2.5)	Windows (10.0.2.15)	At 14:01 UTC	To see what the endpoint exposes	All 1000 ports are in ignored states; no services exposed	
Port scan of ubuntu	Kali (10.0.2.5)	Ubuntu (10.0.2.4)	At 13:58 UTC	To see what the server exposes	Port 22/tcp open, ssh	sshd error from 10.0.2.5 at 13:58 UTC in auth.log
Controlled failed SSH logins, then a correct login	Kali (10.0.2.5)	Ubuntu (10.0.2.4), port 22	17:52:21 UTC	To create a failed and successful login evidence in a safe way	Failed password lines from 10.0.2.5 then an accepted login	Auth.log

Fig 8: Nmap scan of ubuntu and windows, the failed and successful SSH logins on kali



09 Before vs After

We followed baseline, change, evidence, investigation, and kept what we observed separate from what we concluded.

Area	Baseline (before)	My activity (after)	What we observed	Conclusion
Ubuntu SSH	Only sudo use and normal logins in auth.log	Nmap scan and failed SSH logins from kali	An sshd error from 10:0:2:5 at 13:58:24 UTC and failed password lines	The activity came from Kali's address, so it was our test; we cannot say what the scan was looking for from the log alone
Windows exposure	Firewall on; ping allowed by our rule	Nmap -Pn scan from Kali	All 1000 ports in ignored states; host still answers ping	The firewall hides services but not the fact that the host is alive
Network traffic	One background IPv6 packet	Pings between the VMs	105 packets with ICMP and ARP between	The machines can talk to each other and

			all three machines	Wireshark shows who did
Ubuntu ports	SSH enabled by design	Nmap -sV scan	Confirms port 22 open	Only the service we enabled is exposed

10. Mini Investigation

Step	Finding
Observation	At 13:58:24 UTC sshd logged a connection from 10.0.2.5, port 36380, that closed before any login. The log also recorded three failed password attempts for blue-beetle from the same address
Evidence	/var/log/auth.log. the Wireshark capture and the inventory identify 10.0.2.5, MAC 08:00:27:5a:87:bc as Kali.
Interpretation	A connection with no login, then repeated failures, then one address fits a scan followed by password guessing that eventually worked.
Alternative explanation	The closed connection could come from a harmless client or a health check, and the failures could be someone mistyping a password. The pattern is a reason to look closer, not a proof of attack.
Conclusion	Here the activity is our own test from Kali, so it is explained. In a real incident the same pattern would justify treating the account as possibly compromised.

11. Recommendations

. Harden SSH

. Limit repeated failures

. Alert on the pattern

. Keep the windows firewall on and review the ICMP exception; allow ping only Where it is needed

. Keep time consistent

. Audit file changes

. patch and maintain

12 Conclusion

We built and documented a three-machine lab, baselined the endpoint, administered the server, captured network traffic, and ran a controlled test that we found and explained in the logs. The scan and the repeated SSH login were visible in the ubuntu authentication log and traceable to Kali's address, while the windows firewall kept the endpoint's services hidden. This exercise showed us how a baseline, consistent clocks, and a few simple commands let an analyst reconstruct what happened and decide what visibility is needed centrally.

Lab Verification

Team member	Role	Lab confirmed	Connectivity evidence
Glory	Team lead	Windows 10, Ubuntu 22.04.5 and Kali 2026.2 on one NAT Network	Kali pinging ubuntu (10.0.2.4) and windows (10.0.2.15) with replies and no loss
Anselm	Assistant team lead	Windows, Kali and Ubuntu on one NAT Network	Kali to ubuntu ping (192.168.56.104 to 192.168.56.101) with replies and zero loss, DNS lookup, port 22 open, SSH login
Ridwanu	Member	Windows 10, Ubuntu 26.04.1, Kali Linux on one NAT Network	Windows (10.0.2.15) to ubuntu (10.0.2.6) and Kali (10.0.2.5) to ubuntu (10.0.2.6) four replies each with zero loss
Aminat	Member	Windows, Kali and Ubuntu	Kali to Ubuntu ping (192.168.56.103) to (192.168.56.102) with 3 packets

			transmitted and 0% packet loss